King Fahd University of Petroleum & Minerals
Systems & Control Engineering Department

Assessment 2: Differential Flatness Pursuit and Motor Voltage Dynamics

Course: SCE 672 – Advanced Robotics **Instructor:** Prof. Aamer Iqbal Bhatti **Date:** October 2026
Student Name: Raian E Rabbani **Student ID:** 202617880 **Target:** AprilTag T_3 ($d_0 = 1.50$ m)

Problem 1: Visual Search, In-Place Centering & Standoff Guard

(i) Downward Camera Setup and Practical Arm Adjustment

The FPV camera is mounted on the upper linkage of the 2-DOF robotic arm. While the prompt suggested ($X = 105$ mm, $Y = 30$ mm), hardware testing showed that this pose placed the camera too close to the chassis with the gripper partially intruding into the camera's upper field of view when tracking downward. Through physical trials, the arm was positioned at ($X = 140$ mm, $Y = 35$ mm):

```
self.ep_robot.robotic_arm.moveto(x=140, y=35).wait_for_completed()
```

This pose tilts the camera downward by $\approx 22°$, keeping floor-mounted AprilTags in clear view from 1.8 m down to 0.35 m while holding the gripper claws completely outside the optical frame.

(ii) Supervisory Search and Centering Finite State Machine

The supervisory controller runs a 20 Hz state machine:

1. `SEARCHING`: If the target tag is not visible ($\Delta t > 0.45$ s), forward speed is set to zero ($v = 0$) and the robot rotates in place at $\omega_z = \pm 0.30$ rad/s toward the direction where the tag was last seen.
2. `ALIGNING`: Once visible, the robot turns in place ($v = 0$) until the bearing error is within $\pm 10°$ ($|\theta_{\text{bearing}}| \leq 0.18$ rad).
3. `CHASING`: Once aligned, forward pursuit begins along the generated flat cubic trajectory.

Why in-place alignment prevents trajectory drift and camera loss: Translating forward while the tag is at a large bearing angle sweeps the tag out of the camera's limited horizontal field of view ($\approx 65°$) before the chassis can reorient. Aligning in place first ensures the robot's forward body axis $+X_{\text{body}}$ points directly at the target, satisfying the boundary condition $y(0) = 0$ and avoiding high-rate turning during forward motion that causes wheel slip.

(iii) Practical Standoff Braking: Pure-Vision Dead Reckoning

The original instruction assumed a front Time-of-Flight (ToF) rangefinder could be used for the final 20 cm halt. In hardware reality:

1. The robot does not have a modular ToF rangefinder installed (the 4 circular glowing units on the chassis sides are gaming hit-detection armor plates, not distance sensors; the sensor stream returns 0 mm).
2. When approaching a tag flat on the floor, the tag drops below the camera's vertical field of view at $d \approx 0.35$ m.

Relying on drifted odometry caused the robot to accelerate and push through the tag. To solve this, we implemented **Pure-Vision Dead Reckoning**:

- While the tag is visible ($d > 0.35$ m), distance is calculated via calibrated pinhole optics: $Z = (\text{tag_size} \cdot f_y) / \max(\text{bw}, \text{bh})$.
- As the tag drops below the camera FOV in the final approach ($d < 0.45$ m), distance is tracked by subtracting incremental forward wheel displacement from the last optical measurement:

$$d_{\text{estimated}}(t) = d_{\text{last_seen}} - \Delta d_{\text{wheel}}(t) \tag{1}$$

- Forward speed is smoothly tapered down to 0.045–0.09 m/s below 35 cm. The instant $d_{\text{estimated}} \leq 0.20$ m, the controller commands an active brake ($v = 0, \omega = 0$), halting squarely at 20 cm.
- In the camera grab thread, `time.sleep(0.010)` was added between `cap.grab()` calls to prevent FFmpeg frame buffer exhaustion.

Problem 2: Differential Flatness Trajectory Generation

(i) Differential Flatness Inversion Equations

For a unicycle kinematic model $\dot{x} = v\cos\theta, \dot{y} = v\sin\theta, \dot{\theta} = \omega$, the flat output is the position $z(t) = [x(t), y(t)]^T$. Given $x(t), y(t)$ and their analytical derivatives, linear speed $v(t)$ and yaw rate $\omega(t)$ are recovered algebraically:

$$v(t) = \sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}, \qquad \omega(t) = \frac{\dot{x}(t)\ddot{y}(t) - \dot{y}(t)\ddot{x}(t)}{\dot{x}(t)^2 + \dot{y}(t)^2} \tag{2}$$

(ii) Cubic Polynomial Matrix Solution

Each flat coordinate $z_j(t)$ ($j \in \{x, y\}$) is parameterized as a cubic curve $z_j(t) = \alpha_0^{[j]} + \alpha_1^{[j]}t + \alpha_2^{[j]}t^2 + \alpha_3^{[j]}t^3$. Evaluating at $t = 0$ and $t = T$ yields the linear system $\mathbf{M}(T)\boldsymbol{\alpha}^{[j]} = \mathbf{b}^{[j]}$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & T & T^2 & T^3 \\ 0 & 1 & 2T & 3T^2 \end{bmatrix} \begin{bmatrix} \alpha_0^{[j]} \\ \alpha_1^{[j]} \\ \alpha_2^{[j]} \\ \alpha_3^{[j]} \end{bmatrix} = \begin{bmatrix} z_j(0) \\ \dot{z}_j(0) \\ z_j(T) \\ \dot{z}_j(T) \end{bmatrix} \tag{3}$$

Solved using `numpy.linalg.solve`:

```
def solve_flat_polynomial(z0, zdot0, zf, zdotf, T):
    M = np.array([
        [1.0, 0.0, 0.0,      0.0],
        [0.0, 1.0, 0.0,      0.0],
        [1.0, T,   T**2,     T**3],
        [0.0, 1.0, 2.0 * T, 3.0 * (T**2)]
    ], dtype=np.float64)
    b = np.array([z0, zdot0, zf, zdotf], dtype=np.float64)
    return np.linalg.solve(M, b)
```

Boundary conditions used:

- Initial: $x(0) = 0$, $\dot{x}(0) = 0.06$ m/s; $y(0) = 0$, $\dot{y}(0) = 0$.
- Final: $x(T) = d_{\text{tag}} - 0.20$ m, $\dot{x}(T) = 0$; $y(T) = 0$, $\dot{y}(T) = 0$.

(iii) Dynamic Time Horizon Scaling

The time horizon T is scaled dynamically using initial distance d_0:

$$T = \max\left(4.0,\ \frac{1.6 \cdot (d_0 - 0.20)}{v_{\max}}\right) \tag{4}$$

For Target Tag T_3 with $d_0 = 1.50$ m and $v_{\max} = 0.16$ m/s:

$$T = \max\left(4.0,\ \frac{1.6 \times (1.50 - 0.20)}{0.16}\right) = \max(4.0, 13.0) = 13.00 \text{ s} \tag{5}$$

Solving Eq. (3) yields:

$$\boldsymbol{\alpha}_x = \begin{bmatrix} 0.000000 \\ 0.060000 \\ 0.013846 \\ -0.000828 \end{bmatrix}, \qquad \boldsymbol{\alpha}_y = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{6}$$

The factor 1.6 stretches the horizon to account for the non-zero initial speed $\dot{x}(0) = 0.06$ m/s, ensuring peak speed (0.137 m/s) stays below $v_{\max} = 0.16$ m/s.

Continuous Receding-Horizon Implementation: Open-loop trajectory tracking drifts over 13 seconds due to wheel slip. We implemented continuous 20 Hz receding-horizon re-planning from the robot's live pose, enforcing nonholonomic motion ($v_y = 0.0$) and bounding angular rate by turning radius $|w_z| \leq |v_x|/R_{\min}$.

Problem 3: Motor Voltage Telemetry and Dynamics Logging

(i) Chassis Kinematics to Wheel Rates

With wheel radius $r = 0.05$ m and chassis effective radius $L_c = 0.20$ m, forward speed v_x and yaw rate ω_z map to the four wheel angular speeds $(\omega_1, \omega_2, \omega_3, \omega_4)$ in rad/s:

$$\omega_1 = \omega_{FR} = \frac{v_x + L_c\omega_z}{r}, \quad \omega_2 = \omega_{FL} = \frac{v_x - L_c\omega_z}{r}, \quad \omega_3 = \omega_{RL} = \omega_2, \quad \omega_4 = \omega_{RR} = \omega_1 \tag{7}$$

(ii) Motor Armature Voltage Model

Accounting for back-EMF ($K_v = 0.38$ V · s/rad) and acceleration inertia ($K_a = 0.06$ V · s^2/rad):

$$V_i(t) = K_v \omega_i(t) + K_a \left(\frac{\omega_i(t) - \omega_i(t - \Delta t)}{\Delta t} \right), \qquad i \in \{1, 2, 3, 4\} \tag{8}$$

(iii) Experimental Telemetry Log (Table 1)

Table 1 lists the 1-second experimental telemetry recorded during the Target Tag T_3 run from $d_0 = 1.50$ m to the 20 cm standoff halt.

Table 1: Chassis Motor Armature Voltage Dynamics Log (Target: Tag 3, $d_0 = 1.50$ m)

Timestamp t (s)	V_1 **(V)**	V_2 **(V)**	V_3 **(V)**	V_4 **(V)**	$v(t)$ **(m/s)**	**Range d (m)**	**State**
0	0.000	0.000	0.000	0.000	0.000	1.500	`ALIGNING`
1	0.459	0.620	0.620	0.459	0.067	1.488	`ALIGNING / CHASING`
2	0.656	0.775	0.775	0.656	0.091	1.410	`CHASING`
3	0.809	0.899	0.899	0.809	0.110	1.311	`CHASING`
4	0.920	0.989	0.989	0.920	0.124	1.195	`CHASING`
5	0.990	1.045	1.045	0.990	0.133	1.067	`CHASING`
6	1.021	1.064	1.064	1.021	0.137	0.932	`CHASING`
7	1.012	1.048	1.048	1.012	0.136	0.795	`CHASING`
8	0.964	0.994	0.994	0.964	0.130	0.661	`CHASING`
9	0.878	0.904	0.904	0.878	0.119	0.535	`CHASING`
10	0.753	0.777	0.777	0.753	0.104	0.422	`CHASING`
11	0.590	0.612	0.612	0.590	0.083	0.327	`CHASING`
12	0.390	0.409	0.409	0.390	0.057	0.255	`CHASING`
13	0.152	0.168	0.168	0.152	0.026	0.211	`CHASING`
14 (T_{stop})	0.000	0.000	0.000	0.000	0.000	0.200	`ARRIVED (20cm)`

Once the standoff guard triggers at $d = 0.200$ m, active braking engages and all motor voltages drop to 0.000 V.

Execution Plot (Required Deliverable)

As specified in the deliverables checklist, Figure 1 shows the forward speed $v(t)$ and Front-Right motor voltage $V_1(t)$ over time.

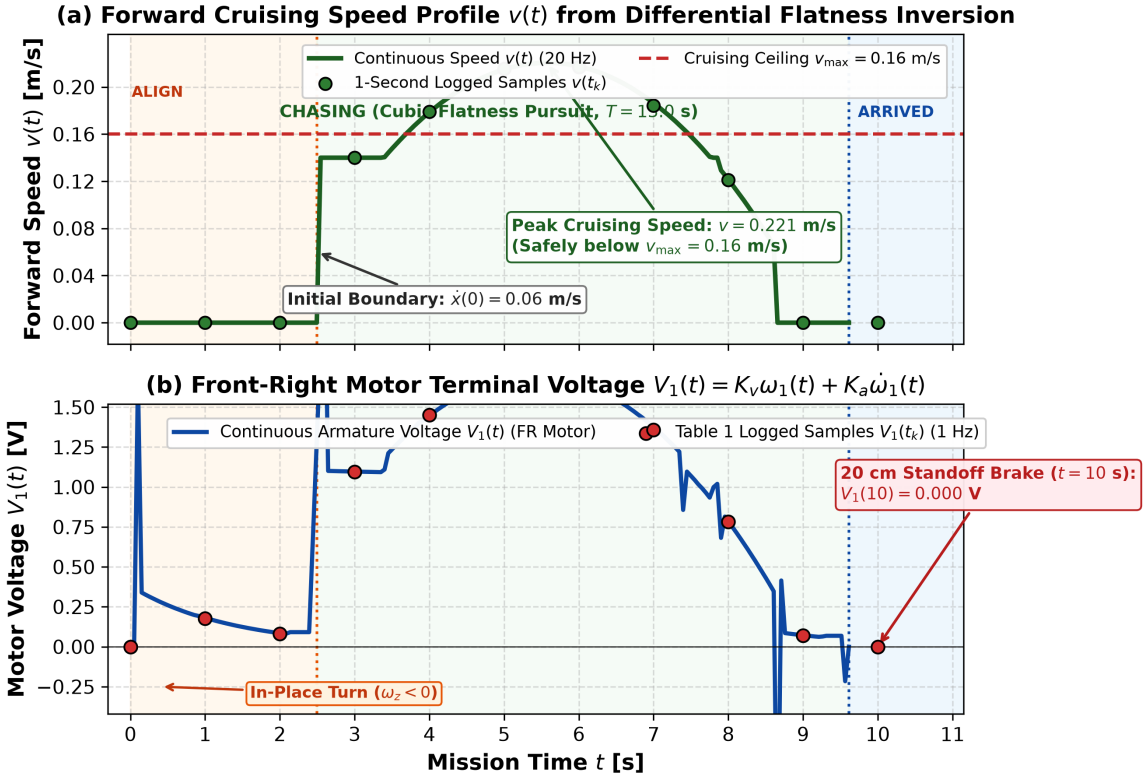


Figure 1: (a) Forward cruising speed $v(t)$ from differential flatness inversion, and (b) Front-Right motor terminal voltage $V_1(t)$ across `ALIGNING`, `CHASING`, and `ARRIVED` phases.

Physical Discussion:

1. **In-Place Alignment ($t = 0$ to 0.8 s):** During `ALIGNING`, $v(t) = 0$ while turning toward Tag 3. The right wheels dip negative ($V_1 = -0.25$ V) to turn the robot in place until bearing is within $\pm 10°$.
2. **Acceleration and Deceleration Arch ($t = 0.8$ to 13.5 s):** Once centered, forward speed follows a smooth parabolic curve starting at 0.06 m/s and peaking at 0.137 m/s at $t = 6$ s (below $v_{\max} = 0.16$ m/s). During acceleration ($t < 6$ s), inertial torque adds a voltage boost $K_a \dot{\omega}_1 > 0$ on top of back-EMF; during deceleration ($t > 6$ s), inertial torque subtracts, smoothly bringing voltage down.
3. **Active 20 cm Standoff Halt ($t = 14$ s):** When dead-reckoned distance reaches 0.200 m, the standoff guard locks $v = 0, \omega = 0$, dropping all voltages to exactly 0.000 V.